React

Build Modern Web Applications

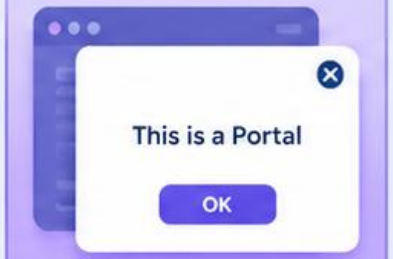


Course Outline

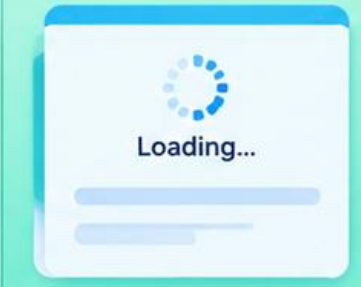
1. **React Forms**



2. **React Portals**



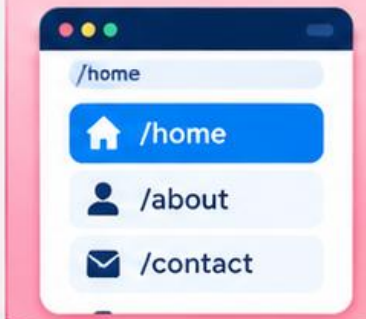
3. **React Suspense**



4. **Styling React**



5. **React Router**



6. **React Hooks**



React Forms

❖**What are React Forms?**

- React Forms are used to collect, manage, and process user input within React applications.

❖**Key Points**

✓Forms allow users to enter data.

✓Used for registration, login, search, feedback, and data entry.

✓React manages form data using state.

✓Forms can contain different input elements.

✓React provides **controlled** and **uncontrolled** forms.

Controlled and Uncontrolled Form

The difference is **who owns the input's value**: React state or the DOM.

Controlled

React state is the single source of truth. The input's `value` comes from state, and every change updates that state.

jsx

```jsx
import { useState } from "react";

function ControlledForm() {
  const [email, setEmail] = useState("");

  const handleSubmit = (e) => {
    e.preventDefault();
    console.log(email);
  };

  return (
    <form onSubmit={handleSubmit}>
      <input
        type="email"
        value={email}
        onChange={(e) => setEmail(e.target.value)}
      />
      <button type="submit">Send</button>
    </form>
  );
}
```

Uncontrolled

The DOM keeps the value, as in plain HTML. You read it only when you need it, usually through a `ref` or `FormData`.

jsx

```jsx
import { useRef } from "react";

function UncontrolledForm() {
  const emailRef = useRef(null);

  const handleSubmit = (e) => {
    e.preventDefault();
    console.log(emailRef.current.value);
  };

  return (
    <form onSubmit={handleSubmit}>
      <input type="email" defaultValue="" ref={emailRef} />
      <button type="submit">Send</button>
    </form>
  );
}
```

Note the `defaultValue` instead of `value`. It sets only the initial value.

Cont. ...

Key differences

	Controlled	Uncontrolled
Value stored in	React state	The DOM
Read value via	State variable	`ref` or `FormData`
Initial value	`value`	`defaultValue`
Re-renders on each keystroke	Yes	No
Live validation, conditional UI	Easy	Harder
Code amount	More	Less

Which to use

- **Controlled:** live validation, disabling the submit button until the form is valid, formatting input as the user types, or fields that depend on each other. This is the usual choice for most forms.
- **Uncontrolled:** simple forms where you only need values on submit, file inputs (`<input type="file">` is always uncontrolled), or integrating with non-React code.

For large forms, libraries like **React Hook Form** use uncontrolled inputs under the hood to avoid re-rendering on every keystroke, while still giving you validation and a clean API.

React State Vs DOM

- **React state** is data that React stores and manages inside a component. When state changes, React re-renders the component so the UI matches it. In a controlled form, state is the single source of truth: the input's value comes from state, and every keystroke updates it through onChange.
- **The DOM (Document Object Model)** is the browser's own representation of the page. DOM elements, including form inputs, keep their own internal values. In an uncontrolled form, React leaves the value in the DOM and reads it only when needed, using a ref or FormData.
- Controlled inputs store their value in React state, which gives you full control and live validation. Uncontrolled inputs store their value in the DOM, which means less code and fewer re-renders.

Cont. ...

Example

JavaScript

```
function App() {
  return (
    <form>
      <input
        type="text"
        placeholder="Enter Name"
      />
      <button type="submit">
        Submit
      </button>
    </form>
  );
}
```

Output

```
[ Enter Name ]

[ Submit ]
```

Why Do We Need React Forms?

Key Points

- Collect user information.
- Validate input data.
- Process user requests.
- Send data to servers or APIs.
- Create interactive applications.

Common Examples

- User Registration
- Login Forms
- Contact Forms
- Search Forms
- Feedback Forms

How React Forms Work

Process

1. User enters data.
2. React captures input values.
3. State stores the form data.
4. User submits the form.
5. React processes or sends the data.

Flow

```
User Input
     ↓
React State
     ↓
Form Submission
     ↓
Processing / API
```

Subtopics of React Forms

1. Controlled Components

Explanation

A controlled component is an input element whose value is controlled by React state.

Features

- React manages input data.
- Single source of truth.
- Easier validation and control.

Cont. ...

Example

JavaScript

```
import { useState } from "react";

function App() {
  const [name, setName] =
    useState("");

  return (
    <input
      type="text"
      value={name}
      onChange={(e) =>
        setName(e.target.value)
      }
    />
  );
}
```

2. Handling Input Fields

Explanation

React captures user input using the `onChange` event.

Example

JavaScript

```
function App() {
  const handleChange = (e) => {
    console.log(e.target.value);
  };

  return (
    <input
      type="text"
      onChange={handleChange}
    />
  );
}
```

3. Form Submission

Explanation

The `onSubmit` event handles form submission.

Example

JavaScript

```
function App() {
  const handleSubmit = (e) => {
    e.preventDefault();

    alert("Form Submitted");
  };

  return (
    <form onSubmit={handleSubmit}>
      <button type="submit">
        Submit
      </button>
    </form>
  );
}
```

Cont. ...

4. Multiple Input Fields

Explanation

A form can contain multiple input fields managed by React.

Example

JavaScript

```
import { useState } from "react";

function App() {
  const [formData, setFormData] =
    useState({
      name: "",
      email: ""
    });

  return (
    <>
      <input
        type="text"
        placeholder="Name"
      />

      <input
        type="email"
        placeholder="Email"
      />
    </>
  );
}
```

5. Textarea

Explanation

Used for entering multiple lines of text.

Example

JavaScript

```
function App() {
  return (
    <textarea
      rows="4"
      cols="30"
    />
  );
}
```

Output

```
Multi-line text area
```

6. Select Dropdown

Explanation

Allows users to choose one option from a list.

Example

JavaScript

```
function App() {
  return (
    <select>
      <option>React</option>
      <option>Angular</option>
      <option>Vue</option>
    </select>
  );
}
```

Output

▼ React

Cont. ...

7. Radio Buttons

Explanation

Allow users to select one option from multiple choices.

Example

JavaScript

```
function App() {
  return (
    <>
      <input
        type="radio"
        name="gender"
      />
      Male

      <input
        type="radio"
        name="gender"
      />
      Female
    </>
  );
}
```

8. Checkboxes

Explanation

Allow users to select multiple options.

Example

JavaScript

```
function App() {
  return (
    <>
      <input type="checkbox" />
      React

      <input type="checkbox" />
      Node.js
    </>
  );
}
```

9. Form Validation

Explanation

Validation ensures users enter correct data.

Example

JavaScript

```
function App() {
  const handleSubmit = (e) => {
    e.preventDefault();

    if (!name) {
      alert("Name Required");
    }
  };
}
```

Cont. ...

10. Uncontrolled Components

Explanation

Inputs manage their own state using refs instead of React state.

Example

JavaScript

```
import { useRef } from "react";

function App() {
  const inputRef = useRef();

  const handleSubmit = () => {
    alert(inputRef.current.value);
  };

  return (
    <>
      <input ref={inputRef} />

      <button onClick={handleSubmit}>
        Submit
      </button>
    </>
  );
}
```

Controlled vs Uncontrolled Components

Controlled	Uncontrolled
Uses State	Uses Ref
React controls value	DOM controls value
Easier validation	Simpler for small forms
Preferred in React	Less commonly used

Cont. ...

Common Mistakes

Mistake 1: Forgetting onChange

❌ Incorrect

JavaScript

```
<input
  value={name}
/>
```

Problem

Input becomes read-only.

Correct

JavaScript

```
<input
  value={name}
  onChange={(e) =>
    setName(e.target.value)
  }
/>
```

Mistake 2: Forgetting preventDefault()

❌ Incorrect

JavaScript

```
const handleSubmit = () => {
  alert("Submitted");
};
```

Problem

The page refreshes after submission.

Correct

JavaScript

```
const handleSubmit = (e) => {
  e.preventDefault();
};
```

Mistake 3: Missing name Attribute

❌ Incorrect

JavaScript

```
<input type="radio" />
<input type="radio" />
```

Problem

Both radio buttons can be selected.

Correct

JavaScript

```
<input
  type="radio"
  name="gender"
/>

<input
  type="radio"
  name="gender"
/>
```

Example

JSXFormPartOne.jsx M | JSXFormPartOne.css M

src > Component > JSXFormPartOne.jsx > JSXFormPartOne > handleSubmit

```jsx
// Demonstrates a controlled form with basic validation.
import { useState } from "react";
import "./JSXFormPartOne.css";

function JSXFormPartOne() {
  // Keep the input value in React state.
  const [name, setName] = useState("");

  // Validate and handle form submission.
  const handleSubmit = (event) => {
    event.preventDefault();

    if (!name.trim()) {
      alert("Name is required");
      return;
    }

    alert(`Welcome ${name}`);
  };

  return (
    <div className="form-page">
      <div className="form-card">
        <h1 className="form-title">Basic React Form</h1>
        <p className="form-subtitle">Enter your name and press Submit</p>

        <form className="form-body" onSubmit={handleSubmit}>
```

JSXFormPartTwo.jsx M | JSXFormPartTwo.css M

src > Component > JSXFormPartTwo.jsx > JSXFormPartTwo

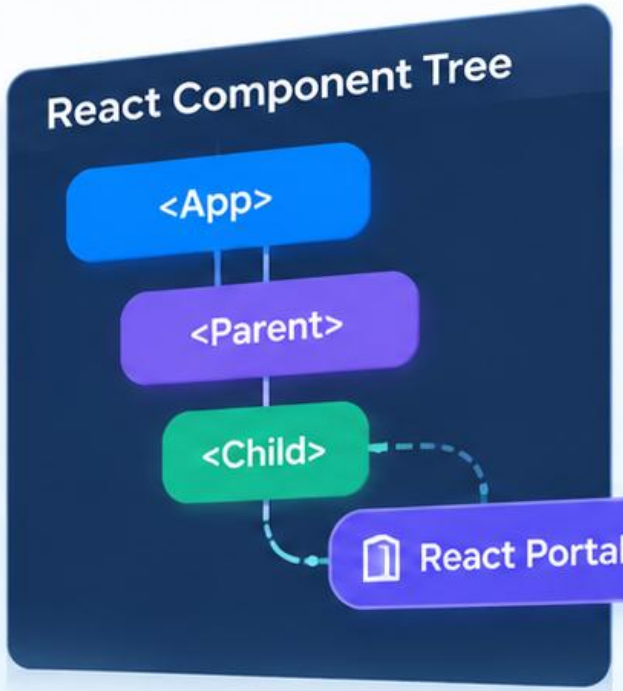
```jsx
function JSXFormPartTwo() {
                <textarea
                  className="reg-textarea"
                  id="address"
                  name="address"
                  rows="3"
                  value={formData.address}
                  onChange={handleChange}
                  placeholder="City, Street, Postal Code"
                />
              </div>
            </fieldset>

            <button className="reg-submit" type="submit">
              Register Student
            </button>
          </form>

          <details className="reg-preview">
            <summary>Live form data (click to expand)</summary>
            <pre>{JSON.stringify(formData, null, 2)}</pre>
          </details>
        </div>
      </div>
    );
}
```

React Portals

Render children into a DOM node outside the parent component

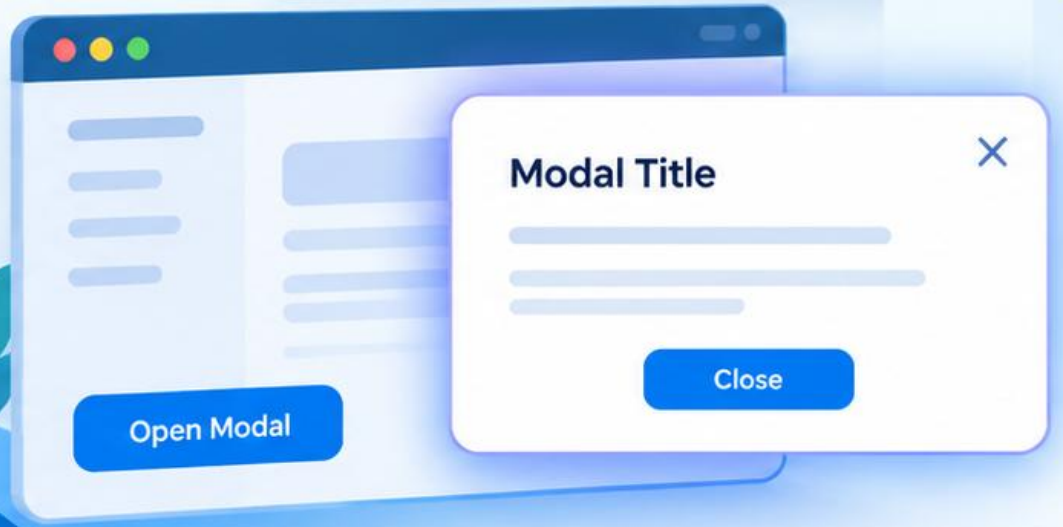


Parent DOM (in-place)

```
<div id="root">
  <div class="app">
    <div class="parent">
      <div class="child">
        <button>Open Modal</button>
      </div>
    </div>
  </div>
</div>
```

Portal DOM (outside parent)

```
<div id="modal-root">
  <div class="modal">
    <h2>Modal Title</h2>
    <p>This is rendered via a portal!</p>
    <button>Close</button>
  </div>
</div>
```



Keep your UI flexible and clean!

React Portals

❖ **What are React Portals?**

- React Portals provide a way to **render a component outside its parent DOM** hierarchy while still maintaining its position in the React component tree.

❖ **Key Points**

✓ Render components outside the root DOM node.

✓ Useful for modals, popups, tooltips, and notifications.

✓ Maintain React features such as props, state, and event handling.

✓ Improve UI layering and positioning.

✓ Prevent CSS overflow and z-index issues.

Cont. ...

Why Do We Need React Portals?

Key Points

- Display modal dialogs above all content.
- Create tooltips that escape parent containers.
- Show notifications anywhere on the page.
- Avoid CSS overflow issues.
- Manage z-index more effectively.

Problem Without Portal

```
Parent Container
  ├── overflow: hidden
  └── Modal
```

The modal may be clipped by the parent container.

Solution With Portal

```
Parent Container
        ↓
Portal
        ↓
Modal Rendered Outside Parent
```

The modal appears correctly above the entire page.

Cont. ...

How React Portals Work

Process

1. Create a target DOM element.
2. Use `ReactDOM.createPortal()`.
3. Pass JSX content.
4. Specify the target DOM node.
5. React renders content outside the parent hierarchy.

Syntax

JavaScript

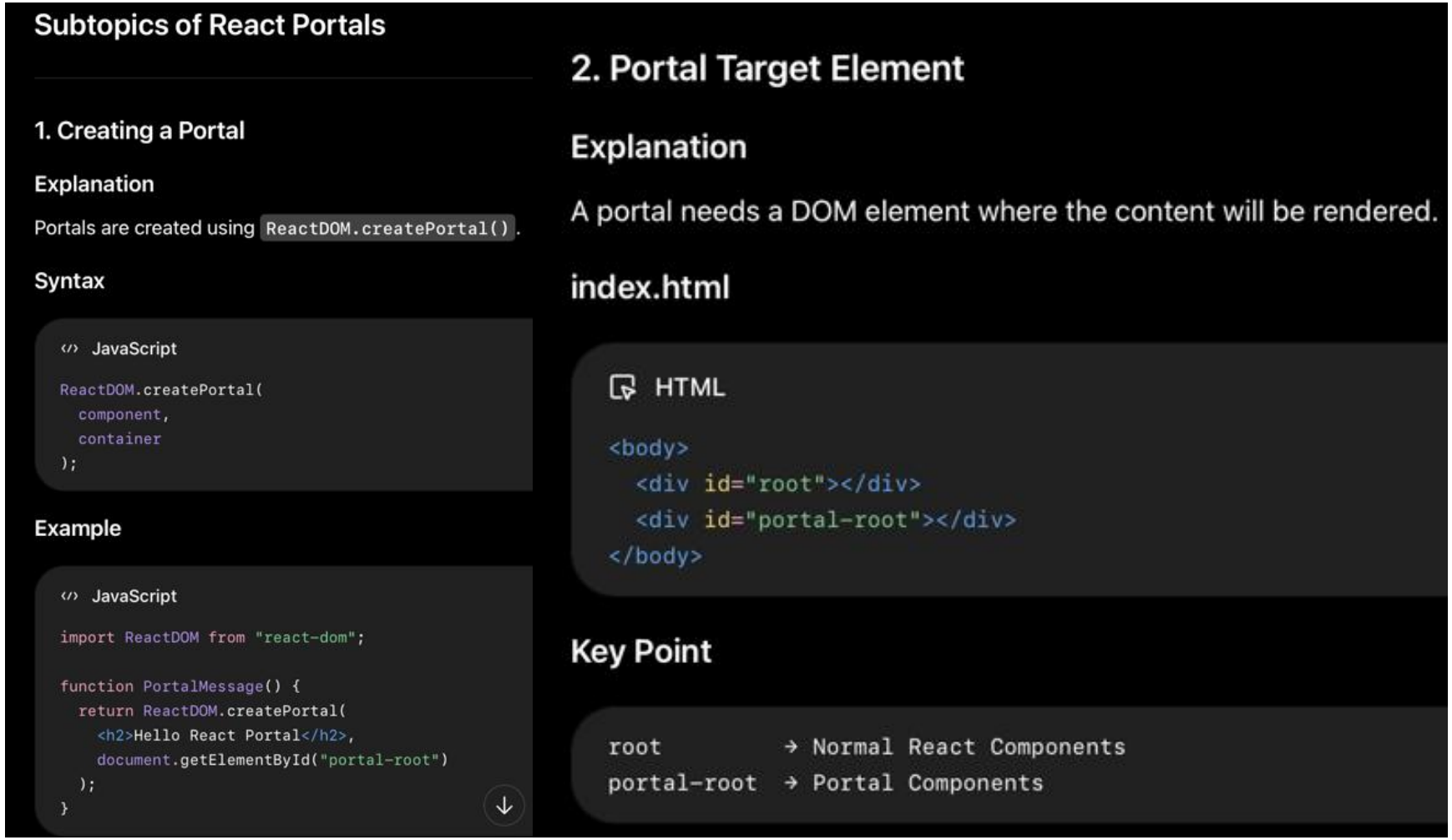
```
ReactDOM.createPortal(
  JSX,
  targetDOMNode
);
```

Example

JavaScript

```
ReactDOM.createPortal(
  <h2>Portal Content</h2>,
  document.getElementById("portal-root")
);
```

Cont. ...


Subtopics of React Portals
1. Creating a Portal
Explanation
Portals are created using ReactDOM.createPortal().
Syntax
JavaScript
ReactDOM.createPortal(
component,
container
);
Example
JavaScript
import ReactDOM from "react-dom";
function PortalMessage() {
return ReactDOM.createPortal(
<h2>Hello React Portal</h2>,
document.getElementById("portal-root")
);
}
2. Portal Target Element
Explanation
A portal needs a DOM element where the content will be rendered.
index.html
HTML
<body>

</body>
Key Point
root → Normal React Components
portal-root → Portal Components

Cont. ...

3. Modal Using Portal

Explanation

The most common use of portals is creating modal dialogs.

Example

JavaScript

```
import ReactDOM from "react-dom";

function Modal() {
  return ReactDOM.createPortal(
    <div>
      <h2>Modal Window</h2>
    </div>,
    document.getElementById("portal-root")
  );
}
```

4. Tooltip Using Portal

Explanation

Tooltips often need to appear outside parent containers.

Example

JavaScript

```
function Tooltip() {
  return ReactDOM.createPortal(
    <div>
      Student Details
    </div>,
    document.getElementById("portal-root")
  );
}
```

Cont. ...

5. Notification Using Portal

Explanation

Notifications can be displayed globally using portals.

Example

JavaScript

```
function Notification() {
  return ReactDOM.createPortal(
    <div>
      Registration Successful
    </div>,
    document.getElementById("portal-root")
  );
}
```

7. Styling Portal Components

Explanation

Portal content can be styled like any React component.

Example

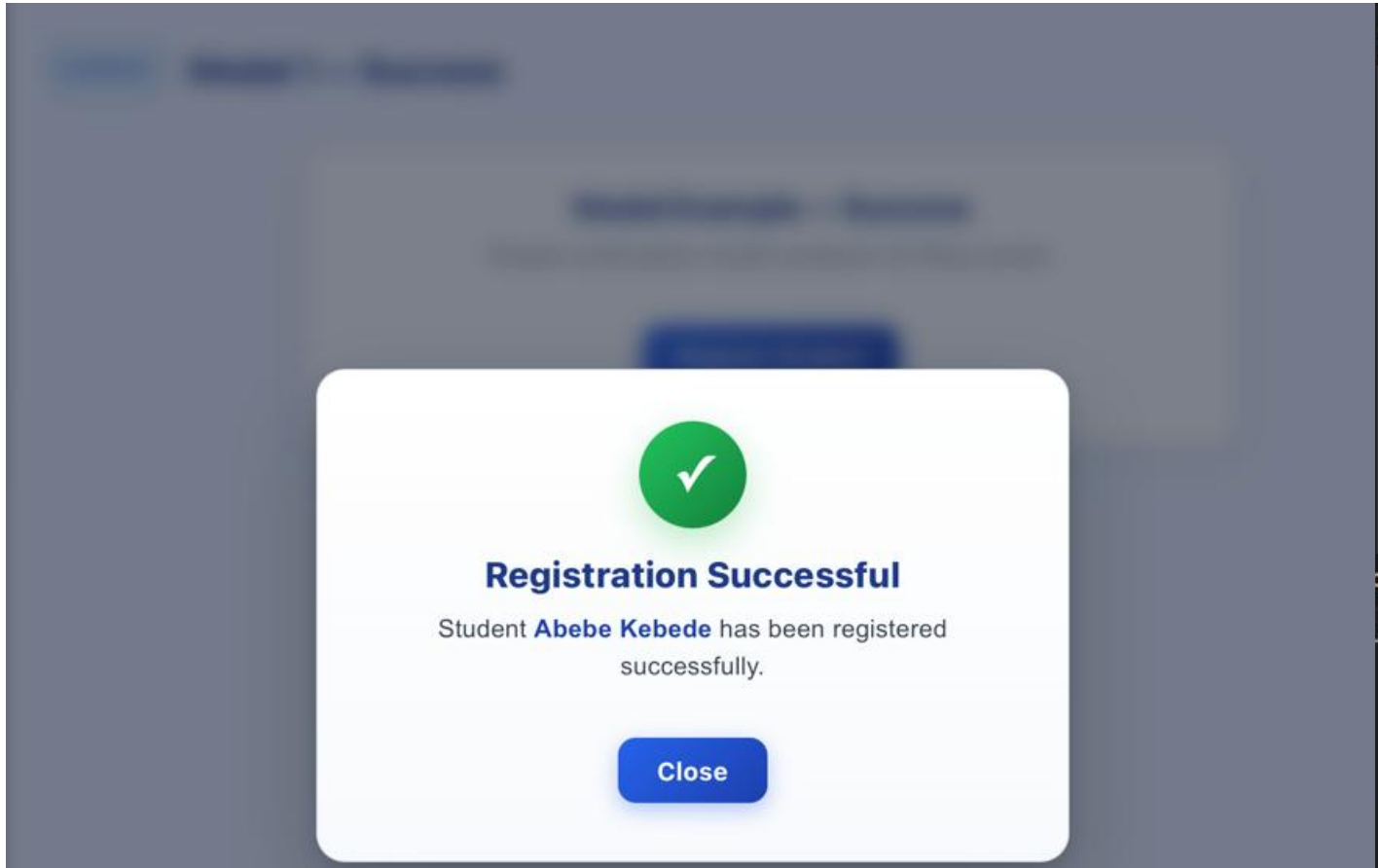
CSS

```
.modal {
  position: fixed;
  top: 30%;
  left: 40%;
}
```

Benefit

- Better positioning.
- Overlay effects.
- Responsive UI.

Example




```jsx
// ============================================================
function ModalExample2({ student, onClose, onConfirm }) {
  // ---------- State ----------
  const [typed, setTyped] = useState(""); // text typed by user
  const [busy, setBusy] = useState(false); // loading state
  const [errorMsg, setErrorMsg] = useState(""); // error message

  // ---------- Refs ----------
  const inputRef = useRef(null); // for auto-focus

  // ---------- Effect: focus input + lock scroll + Esc key ----------
  useEffect(() => {
    // Focus the confirmation input when the modal opens.
    inputRef.current?.focus();

    // Lock page scroll while modal is open (common real-world detail).
    const prevOverflow = document.body.style.overflow;
    document.body.style.overflow = "hidden";
```

.jsx M | # App.css | index.html × | ModalExample2.jsx | #

_App > index.html > ...

```html
<!doctype html>
<html lang="en">
  <body>
    <div id="root"></div>
    <script type="module" src="/src/main.jsx"></script>
    <div id="portal-root"></div>
  </body>
</html>
```

React Suspense

❖**What is React Suspense?**

- React Suspense allows React to temporarily show fallback content while some part of the UI is not ready to render.

❖**Key Points**

✓Used for loading states.

✓Works well with lazy-loaded components.

✓Displays fallback UI while content is loading.

✓Improves user experience.

✓Helps reduce initial application loading time.

✓Commonly used with React.lazy().

Example

JavaScript

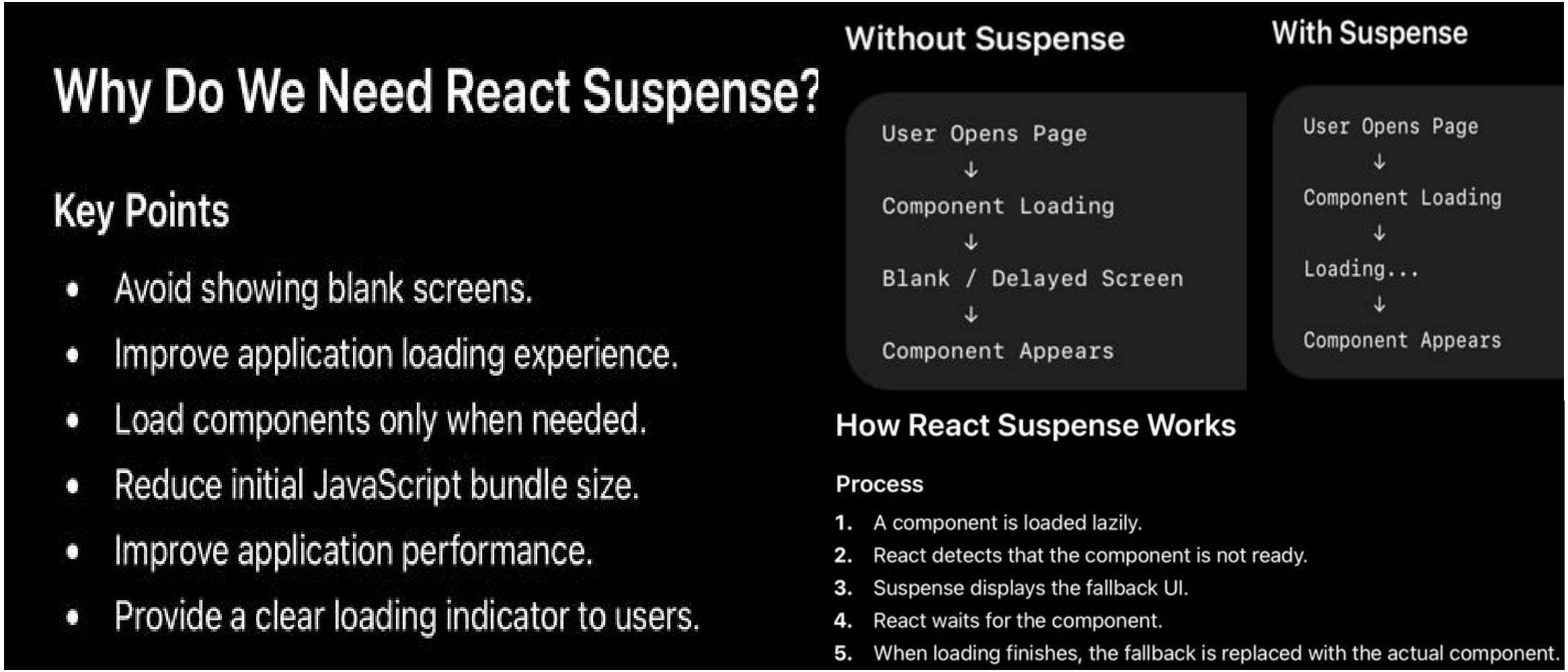
```
import { Suspense } from "react";

<Suspense fallback={<p>Loading...</p>}>
  <Dashboard />
</Suspense>
```

While `Dashboard` is loading:

```
Loading...
```

Cont. ...


Why Do We Need React Suspense?
Key Points
Avoid showing blank screens.
Improve application loading experience.
Load components only when needed.
Reduce initial JavaScript bundle size.
Improve application performance.
Provide a clear loading indicator to users.
Without Suspense
User Opens Page
↓
Component Loading
↓
Blank / Delayed Screen
↓
Component Appears
With Suspense
User Opens Page
↓
Component Loading
↓
Loading...
↓
Component Appears
How React Suspense Works
Process
1. A component is loaded lazily.
2. React detects that the component is not ready.
3. Suspense displays the fallback UI.
4. React waits for the component.
5. When loading finishes, the fallback is replaced with the actual component.

Cont. ...

Subtopics Summary Table

Subtopic	Purpose	Example
Suspense	Define loading boundary	`<Suspense>`
fallback	Display temporary UI	`fallback={<p>Loading...</p>}`
React.lazy()	Load component dynamically	`lazy(() => import())`
Lazy Loading	Load components when needed	`<Dashboard />`
Code Splitting	Divide application bundle	Separate JS chunks
Conditional Loading	Load based on condition	`show && <Dashboard />`
Multiple Components	Load grouped components	Components inside Suspense
Nested Suspense	Separate loading boundaries	Multiple `<Suspense>`

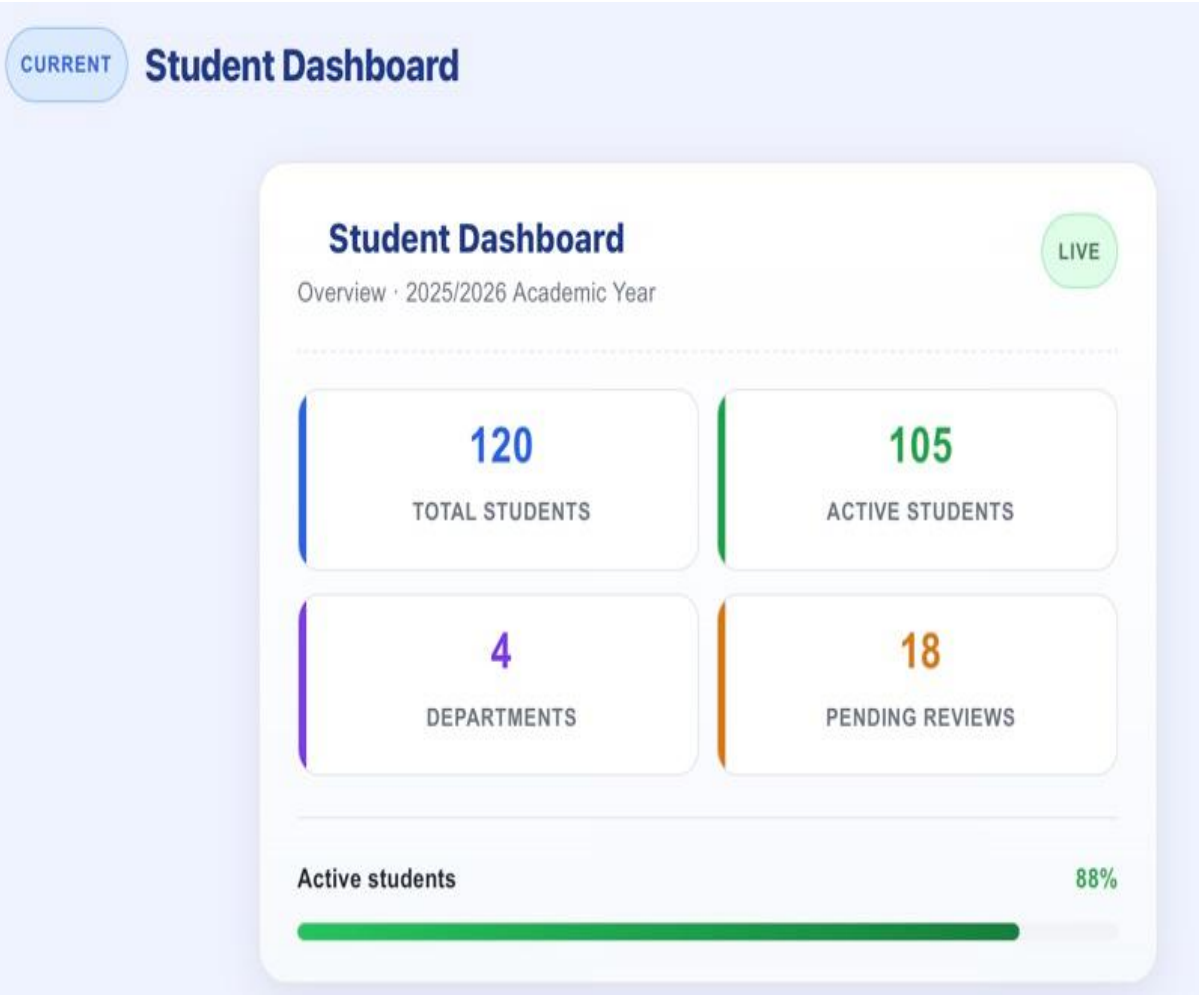
Cont. ...

React_Pro

App.jsx M | StudentDashboard.jsx U

React_App > src > Component > StudentDashboard.jsx > ...

```
46  // ============================================================
47  // StudentDashboard component
48  // ============================================================
49  function StudentDashboard() {
50    // ---------- Derived values ----------
51    // We look up the total and active counts from STATS.
52    // Deriving them here avoids duplicating numbers.
53    const total = STATS[0].value;
54    const active = STATS[1].value;
55
56    // Percentage of active students (used by the progress bar).
57    // Math.round prevents long decimals.
58    const activePercent = Math.round((active / total) * 100);
59
60    return (
61      // Outer wrapper; className links to .dashboard-card in CSS.
62      <div className="dashboard-card">
63        {/* ---------- HEADER: title + subtitle + live badge ---------- */}
64        <header className="dashboard-card__header">
65          <div>
66            <h2 className="dashboard-card__title">Student Dashboard</h2>
67            <p className="dashboard-card__subtitle">
68              Overview · 2025/2026 Academic Year
69            </p>
70          </div>
71          <span className="dashboard-card__badge">Live</span>
```



Styling React Using CSS

❖ **What is Styling in React?**

- Styling in React is the process of applying visual design, layout, colors, spacing, fonts, and responsiveness to React components using CSS.

❖ **Key Points**

✓ Controls the appearance of React applications.

✓ Separates UI design from component logic.

✓ Makes applications more attractive and user-friendly.

✓ Enables responsive and professional layouts.

✓ Supports multiple styling approaches.

Example

JavaScript

```
function App() {
  return (
    <h1 className="title">
      Welcome to React
    </h1>
  );
}
```

CSS

```
.title {
  color: blue;
  text-align: center;
}
```

Cont. ...

Subtopics Summary Table

Subtopic	Purpose	Example
External CSS	Separate styles	`App.css`
className	Apply CSS classes	`className="title"`
Inline CSS	Style in JSX	`style={{}}`
Multiple Classes	Apply multiple styles	`"title active"`
Dynamic Styling	Change styles dynamically	Ternary Operator
CSS Modules	Component-specific styling	`styles.card`
Conditional Styling	Apply styles based on conditions	Pass/Fail
Responsive Design	Mobile-friendly UI	Media Query
Component Styling	Separate CSS per component	`Header.css`

React Router

Navigate Between Pages in a React Application

Build Single Page Apps with Seamless Navigation!

App.jsx

```jsx
import { BrowserRouter, Routes,
  Route, Link } from 'react-router-dom';

function App() {
  return (
    <BrowserRouter>
      <nav className="navbar">
        <Link to="/">Home</Link>
        <Link to="/about">About</Link>
        <Link to="/users">Users</Link>
        <Link to="/settings">Settings</Link>
      </nav>

      <Routes>
        <Route path="/" element={<Home />} />
        <Route path="/about" element={<About />} />
        <Route path="/users" element={<Users />} />
        <Route path="/settings" element={<Settings />} />
      </Routes>
    </BrowserRouter>
);
```











- **Declarative Routing**
- **Dynamic Navigation**
- **Nested Routes**
- **Better User Experience**

React Router

❖ **What is React Router?**

- React Router is a library that enables navigation between different pages (components) in a React application without reloading the entire webpage.

❖ **Key Points**

✓ Provides client-side routing.

✓ Creates Single Page Applications (SPA).

✓ Updates URL without refreshing the browser.

✓ Maps URLs to React components.

✓ Improves user experience with faster navigation.

Cont. ...

❖ **Why Do We Need React Router?**

➢ **Problems Without React Router**

- ✓ Entire page reloads during navigation.
- ✓ Poor user experience.
- ✓ Difficult to manage multiple pages.
- ✓ Increased server requests.

➢ **Benefits of React Router**

- ✓ Fast navigation between pages.
- ✓ No full-page refresh.
- ✓ Cleaner URL management.
- ✓ Better application structure.
- ✓ Supports nested and dynamic routes.

❖ **How React Router Works**

➢ **Routing Process**

1.User clicks a navigation link.

2.URL changes in the browser.

3.React Router matches the route.

4.Corresponding component is rendered.

5.Only required content updates.

How React Router Works

Before writing code, let's understand the routing process — Suppose a user wants to open the Students page.

1

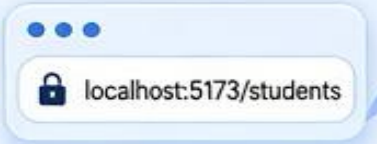


User clicks a navigation link

Example: Clicks Students in the sidebar

- Link: /students

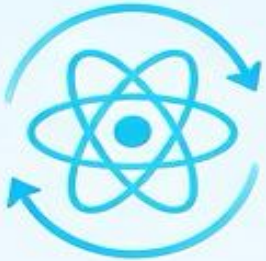
2



Pron in the browser

Address updates to localhost:5173/students

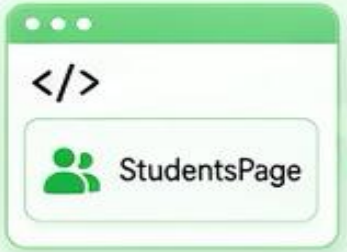
3



React Router matches the route

Checks registered route definitions for a matching path

4



Corresponding component is rendered

StudentsPage is displayed

5



Only the required content updates

App shell and shared UI remain in place

Cont. ...

Slide 4: Installation

Install React Router

‹/› Bash

```
npm install react-router-dom
```

Import Required Components

‹/› JavaScript

```
import {
  BrowserRouter,
  Routes,
  Route
} from "react-router-dom";
```

Slide 5: Core Components of React Router

Important Components

BrowserRouter

- Wraps the entire application.
- Enables routing functionality.

Routes

- Container for all routes.
- Selects matching route.

Route

- Maps URL to component.

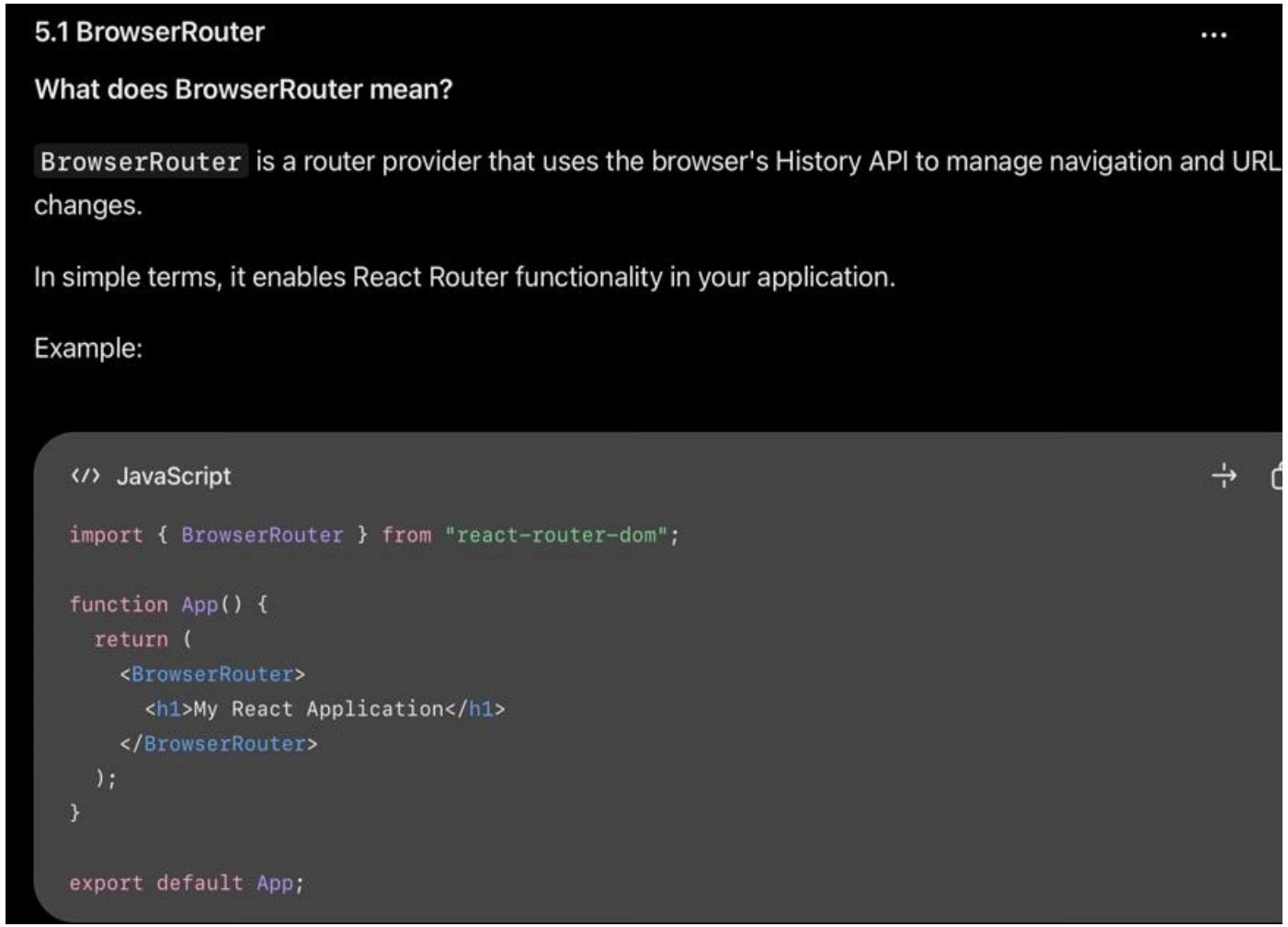
Link

- Navigates without page refresh.

NavLink

- Navigation with active styling.

Basic routing with BrowserRouter, Routes, and Route



❖ **Why do we need BrowserRouter?**

- Without a router provider, the routing components cannot access the routing context they need.
- Think of BrowserRouter as the **foundation of your routing** system.

Routes

5.2 Routes

What does Routes mean?

`Routes` is a container that holds your route definitions.

It examines the current URL and selects the best matching route to render.

Example:

JavaScript

```
<Routes>
  {/* Route definitions go here */}
</Routes>
```

Think of `Routes` as the place where you register the different pages of your application.

Route

5.3 Route

What does Route mean?

`Route` defines the relationship between a URL path and the component that should be displayed.

For example:

JavaScript

```
<Route
  path="/students"
  element={<Students />}
/>
```

This means:

- `path="/students"` : The URL to match.
- `element={<Students />}` : The component to display.

Whenever the user visits `/students` , React Router renders the Students component.

Link

6. Link and NavLink: Navigating between pages

6.1 What is Link?

`Link` is a React Router component used to navigate from one route to another without performing a full browser page reload.

For example, instead of manually typing `/students`, a user can click a Students link.

Example: Navbar.jsx

JavaScript

```
import { Link } from "react-router-dom";

function Navbar() {
  return (
    <nav>
      <Link to="/">Home</Link>

      <Link to="/about">About</Link>

      <Link to="/students">Students</Link>

      <Link to="/courses">Courses</Link>
    </nav>
  );
}

export default Navbar;
```

Let's understand the syntax:

JavaScript

```
<Link to="/students">
  Students
</Link>
```

- `Link` : React Router navigation component.
- `to` : Specifies the destination URL.
- `Students` : Text displayed to the user.

When a user clicks Students, React Router navigates to `/students`.

NavLink

- NavLink is similar to Link, but it provides information about whether the current link is active.
- For example, when the user is viewing the Students page, we may want the Students navigation item to have a different background color.

Cont. ...

Navbar.jsx

```javascript
import { NavLink } from "react-router-dom";

function Navbar() {
  return (
    <nav>
      <NavLink to="/">
        Home
      </NavLink>

      <NavLink to="/students">
        Students
      </NavLink>

      <NavLink to="/courses">
        Courses
      </NavLink>
    </nav>
  );
}

export default Navbar;
```

We can use a function to apply conditional styling.

```javascript
<NavLink
  to="/students"
  className={({ isActive }) =>
    isActive ? "active-link" : "nav-link"
  }
>
  Students
</NavLink>
```

Here:

- `isActive` is true when the link matches the current active route.
- `active-link` is applied to the current page.
- `nav-link` is applied to inactive pages.

Cont. ...

When should we use Link and NavLink?

Link	NavLink
General navigation	Navigation menus
Navigate to another page	Navigate and identify the active page
Does not automatically provide active styling	Provides active-state information

For a sidebar or navbar, `NavLink` is particularly useful.

Understanding route matching

7. Understanding route matching

Route matching is the process React Router uses to determine which route corresponds to the current URL.

Consider these routes:

JavaScript

```
<Routes>
  <Route path="/" element={<Home />} />

  <Route path="/students" element={<Students />} />

  <Route path="/courses" element={<Courses />} />
</Routes>
```

Suppose the current URL is:

Plain text

```
http://localhost:5173/students
```

React Router checks the available routes and finds:

JavaScript

```
<Route
  path="/students"
  element={<Students />}
/>
```

It then renders the Students component.

What happens if the route does not exist?

Suppose the user visits:

Plain text

```
http://localhost:5173/teachers
```

But you have not registered a `/teachers` route.

React Router will not find a matching route.

We should therefore create a Not Found page.

Cont. ...

NotFound.jsx

```javascript
function NotFound() {
  return (
    <div>
      <h1>404 - Page Not Found</h1>

      <p>
        Sorry, the requested page does not exist.
      </p>
    </div>
  );
}

export default NotFound;
```

Register it using a wildcard route:

```javascript
<Routes>

  <Route path="/" element={<Home />} />

  <Route path="/students" element={<Students />} />

  <Route path="/courses" element={<Courses />} />

  <Route path="*" element={<NotFound />} />

</Routes>
```

What does `*` mean?

The wildcard `*` matches paths that do not match any of the other registered routes.

This is useful for handling invalid URLs.

Cont. ...

Slide 8: Navigation Using Link

Example

JavaScript

```
import { Link } from "react-router-dom";

<Link to="/">Home</Link>
<Link to="/about">About</Link>
<Link to="/contact">Contact</Link>
```

Advantages

- No page reload.
- Faster navigation.
- Preserves application state.

Slide 9: Navigation Using NavLink

Example

JavaScript

```
import { NavLink } from "react-router-dom";

<NavLink to="/">
  Home
</NavLink>
```

Benefits

- Detects active route.
- Useful for menus and navigation bars.
- Supports active styling.

Dynamic routing and URL parameters

This is one of the most important routing concepts in real-world applications.

8.1 What is dynamic routing?

Dynamic routing means creating routes that can display different information depending on values provided in the URL.

Consider a university with 1,000 students.

Every student has a unique ID.

For example:

Student	Student ID	URL
Abebe Kebede	101	`/students/101`
Selam Tesfaye	102	`/students/102`
Hana Bekele	103	`/students/103`

Do we need to create a separate React component for each student?

No.

We can create one reusable StudentDetails component and pass the student ID through the URL.

8.2 Define a dynamic route

JavaScript

```
<Route
  path="/students/:id"
  element={<StudentDetails />}
/>
```

Here:

- `/students/` is the fixed part of the URL.
- `:id` is a dynamic parameter.
- The parameter can contain different student IDs.

Examples:

Plain text

```
/students/101
/students/102
/students/103
```

All these URLs match the same route.

Cont. ...

8.3 What is useParams?

`useParams` is a React Router hook that allows a component to access dynamic values from the current URL.

StudentDetails.jsx

```javascript
import { useParams } from "react-router-dom";

function StudentDetails() {

  const { id } = useParams();

  return (
    <div>
      <h1>Student Details</h1>

      <p>Student ID: {id}</p>
    </div>
  );
}

export default StudentDetails;
```

Now, when the user visits:

```
/students/101
```

The output will be:

```
Student Details

Student ID: 101
```

When the user visits:

```
/students/102
```

The output becomes:

```
Student Details

Student ID: 102
```

The same component displays different information based on the URL.

Nested routing

9.1 What is nested routing?

Nested routing means defining routes inside other routes.

It is particularly useful when multiple pages share the same layout or parent component.

Consider a Student Management System with the following structure:

Plain text

```
Dashboard
    |
    ├── Students
    |       ├── Student List
    |       ├── Add Student
    |       └── Student Details
    |
    ├── Courses
    |       ├── Course List
    |       └── Add Course
    |
    └── Enrollments
            ├── Enrollment List
            └── Add Enrollment
```

- The Dashboard layout may contain:
  - ✓ Sidebar
  - ✓ Header
  - ✓ Navigation menu
  - ✓ Footer
- When navigating between Students, Courses, and Enrollments, we do not want to recreate the entire application layout each time.
- Nested routing solves this problem.

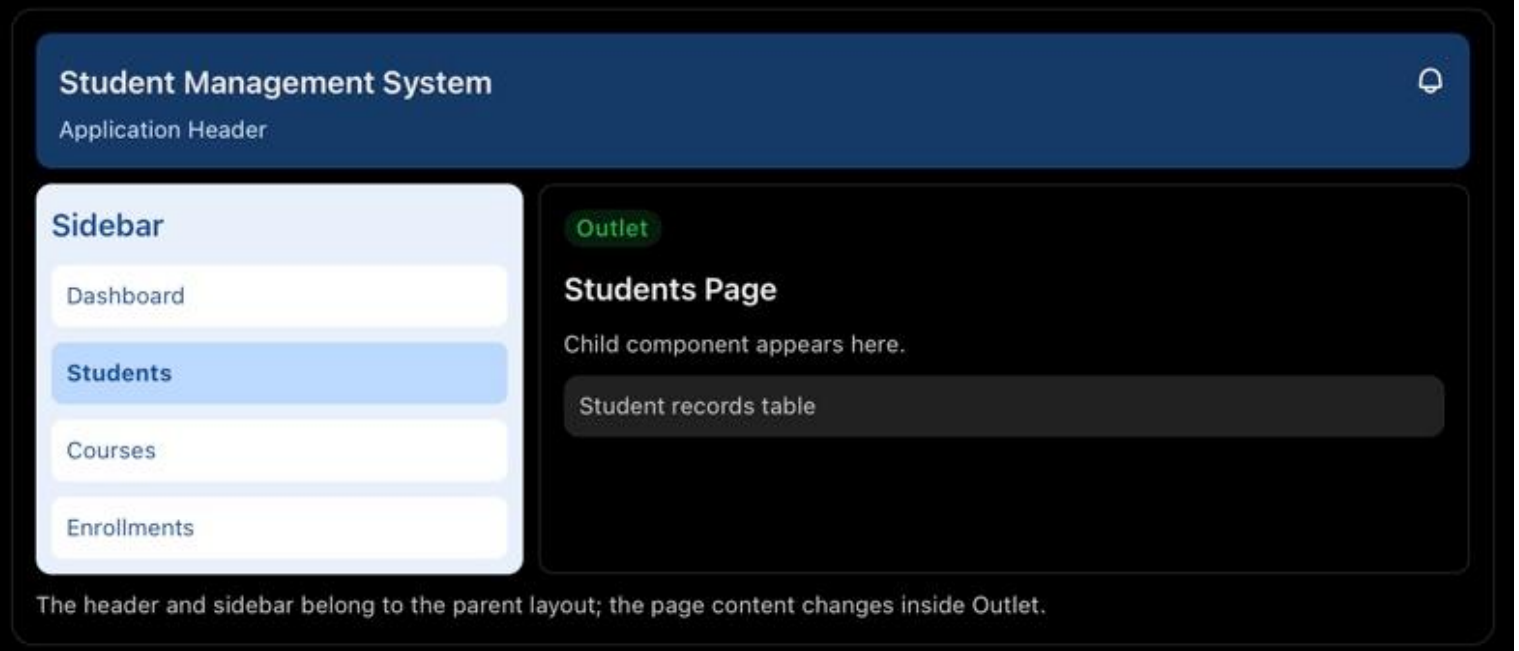
Outlet

9.2 What is Outlet?

`Outlet` is a React Router component that displays the child route component inside its parent route.

Think of `Outlet` as a placeholder.

For example:



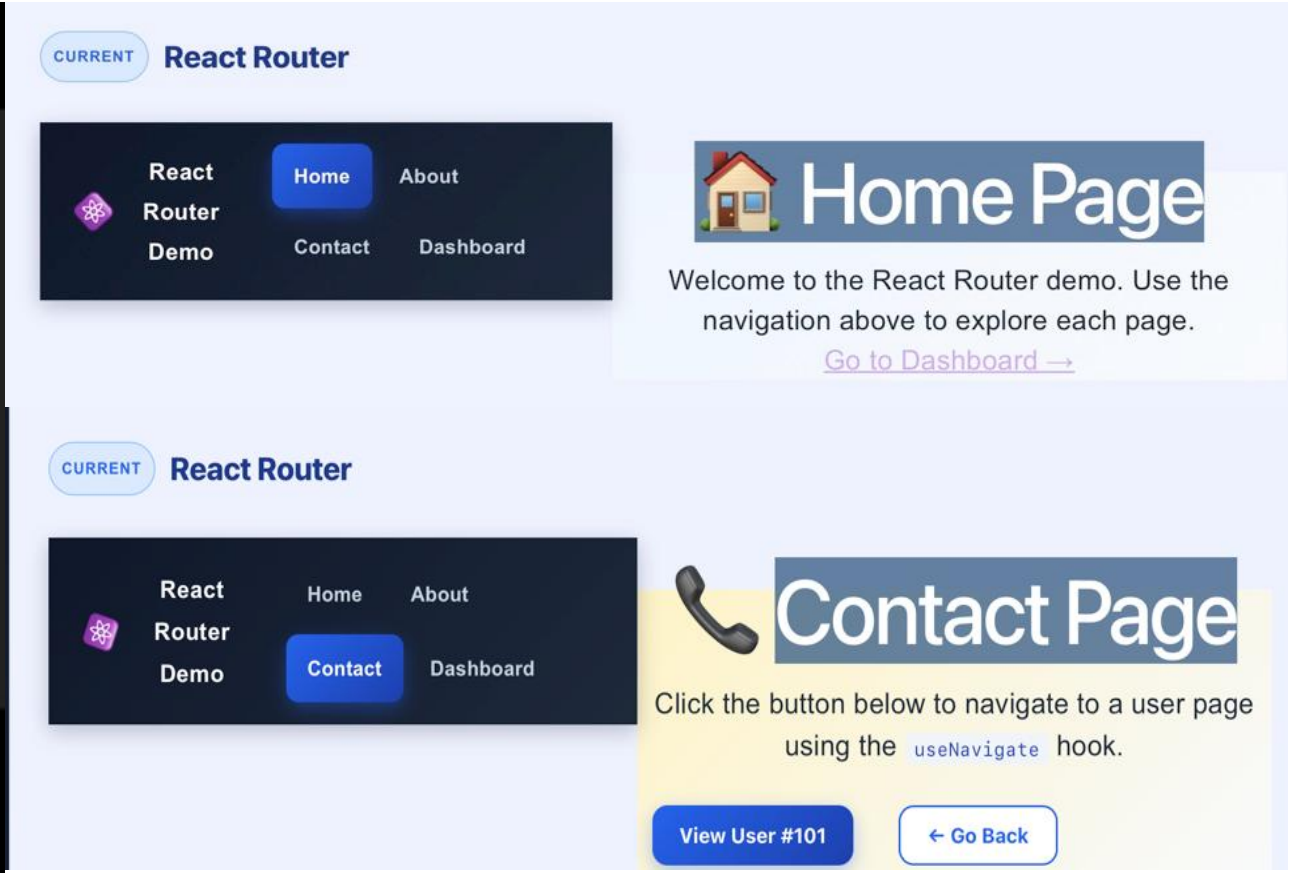
Cont. ...

Slide 16: React Router Folder Structure

```
src
│
├── pages
│   ├── Home.jsx
│   ├── About.jsx
│   └── Contact.jsx
│
├── components
│   └── Navbar.jsx
│
├── App.jsx
└── main.jsx
```

Benefits

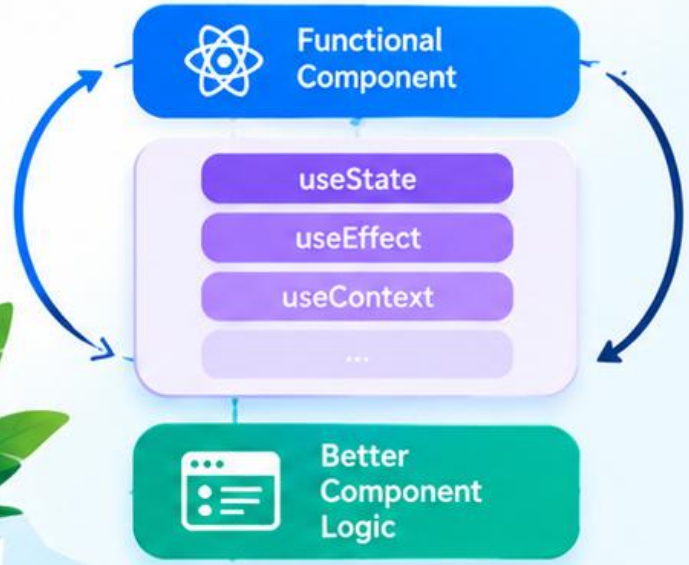
- Organized code.
- Easy maintenance.
- Better scalability.



React Hooks

Use State, Life Cycle and More in Functional Components

Hooks let you use React features without writing class components.





Simple. Powerful. Flexible.

UserProfile.jsx

```jsx
import React, { useState, useEffect } from 'react';
import { useParams } from 'react-router-dom';

function UserProfile() {
  const [user, setUser] = useState(null);
  const [loading, setLoading] = useState(true);
  const { id } = useParams();

  useEffect(() => {
    const fetchUser = async () => {
      const res = await fetch('/api/users/${id}');
      const data = await res.json();
      setUser(data);
      setLoading(false);
    };
    fetchUser();
  }, [id]);

  if (loading) return <p>Loading...</p>;
  return (
    <div>
      <h2>{user.name}</h2>
      <p>{user.email}</p>
    </div>
  );
}
```

Build Smarter Components with React Hooks

Key React Hooks

useState
Adds state to functional components.
Manage component data

useEffect
Handles side effects like API calls, subscriptions, etc.
Run code after render

useContext
Accesses context values without prop drilling.
Share global data

useMemo
Memoizes expensive calculations.
Improve performance

useCallback
Returns a memoized callback function.
Prevent unnecessary re-renders

useRef
Accesses DOM elements or stores mutable values.
Persist values across renders

Hooks make React more powerful!

React Hooks

❖ **React Hooks**

➢ **What are React Hooks?**

- React Hooks are special functions introduced in React 16.8 that allow functional components to use state, lifecycle features, and other React functionalities without writing class components.

❖ **Key Points**

✓ Introduced in React 16.8.

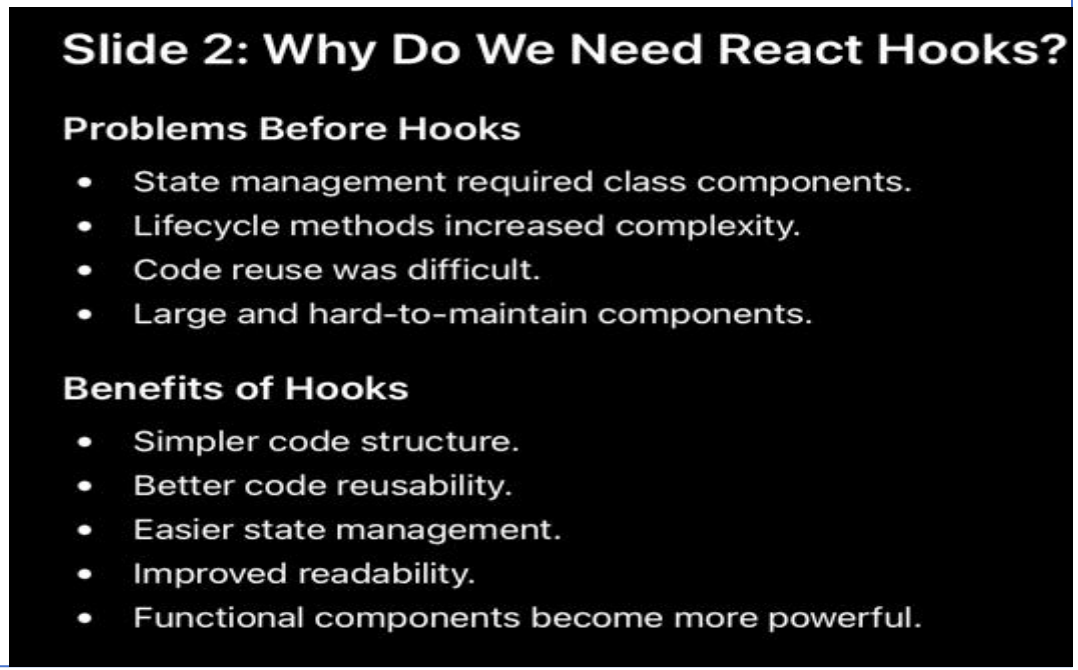
✓ Used inside functional components.

✓ Allow state management.

✓ Enable lifecycle operations.

✓ Make code cleaner and reusable.

✓ Reduce the need for class components.



Cont. ...

Slide 3: How React Hooks Work

Working Process

1. Component renders.
2. Hook initializes state or functionality.
3. User interaction occurs.
4. State updates.
5. Component re-renders with new data.

Flow

```
Component Render
        ↓
Hook Initialization
        ↓
User Interaction
        ↓
State Update
        ↓
Component Re-render
```

Slide 4: Rules of Hooks

Important Rules

- Only call Hooks at the top level.
- Do not call Hooks inside loops.
- Do not call Hooks inside conditions.
- Do not call Hooks inside nested functions.
- Hooks can only be used in:
  - Functional Components
  - Custom Hooks

Correct

JavaScript

```javascript
function App() {
  const [count, setCount] = useState(0);
}
```

Incorrect

JavaScript

```javascript
if (true) {
  useState(0);
}
```

Cont. ...

Slide 5: Common React Hooks

Frequently Used Hooks

Hook	Purpose
useState	Manage state
useEffect	Side effects
useContext	Share data globally
useRef	Access DOM elements
useMemo	Optimize calculations
useCallback	Optimize functions
useReducer	Complex state management
Custom Hook	Reusable logic

Slide 6: useState Hook

What is useState?

Allows components to store and update data.

Syntax

JavaScript

```
const [state, setState] = useState(initialValue);
```

Features

- Stores component data.
- Triggers re-render on update.
- Supports any data type.

Cont. ...

Slide 7: useState Example

Counter Example

JavaScript

```
import { useState } from "react";

function Counter() {
  const [count, setCount] = useState(0);

  return (
    <>
      <h2>Count: {count}</h2>

      <button
        onClick={() =>
          setCount(count + 1)
        }
      >
        Increment
      </button>
    </>
  );
}
```

Slide 8: useEffect Hook

What is useEffect?

Performs side effects after rendering.

Common Uses

- API calls.
- Data fetching.
- Timers.
- Event listeners.
- Local storage operations.

Syntax

JavaScript

```
useEffect(() => {
  // code
}, []);
```

Slide 9: useEffect Example

Component Mount Example

JavaScript

```
import { useEffect } from "react";

function Welcome() {

  useEffect(() => {
    console.log("Component Loaded");
  }, []);

  return <h2>Welcome</h2>;
}
```

Output

```
Component Loaded
```

Cont. ...

Slide 10: useContext Hook

What is useContext?

Provides data to multiple components without passing props manually.

Benefits

- Avoids prop drilling.
- Simplifies data sharing.
- Improves maintainability.

Common Uses

- Theme management.
- Authentication.
- Language settings.

Slide 11: useContext Example

Context Creation

JavaScript

```
import {
  createContext,
  useContext
} from "react";

const UserContext =
  createContext();

function Child() {
  const user =
    useContext(UserContext);

  return <h2>{user}</h2>;
}
```

Cont. ...

Slide 12: useRef Hook

What is useRef?

Stores values without causing re-renders and provides direct access to DOM elements.

Uses

- Focus input fields.
- Store previous values.
- Access DOM elements.

Cont. ...

Slide 13: useRef Example

JavaScript

```
import {
  useRef
} from "react";

function InputFocus() {

  const inputRef = useRef();

  const focusInput = () => {
    inputRef.current.focus();
  };

  return (
    <>
      <input ref={inputRef} />

      <button
        onClick={focusInput}
      >
        Focus Input
      </button>
    </>
  );
}
```

Slide 14: useMemo Hook

What is useMemo?

Optimizes expensive calculations by caching computed values.

Benefits

- Improves performance.
- Prevents unnecessary calculations.
- Useful for large datasets.

Syntax

JavaScript

```
const value = useMemo(
  () => calculation(),
  [dependency]
);
```

Slide 16: useCallback Hook

What is useCallback?

Caches functions to avoid unnecessary recreation.

Benefits

- Improves performance.
- Useful when passing functions to child components.
- Prevents unnecessary re-renders.

Syntax

JavaScript

```
const fn = useCallback(
  () => {},
  []
);
```

Cont. ...

Slide 18: useReducer Hook

What is useReducer?

Alternative to useState for managing complex state logic.

Benefits

- Better for large applications.
- Centralized state updates.
- Predictable state management.

Syntax

JavaScript

```
const [state, dispatch] =
  useReducer(
    reducer,
    initialState
  );
```

Slide 20: Custom Hooks

What is a Custom Hook?

A reusable JavaScript function that uses one or more React Hooks.

Benefits

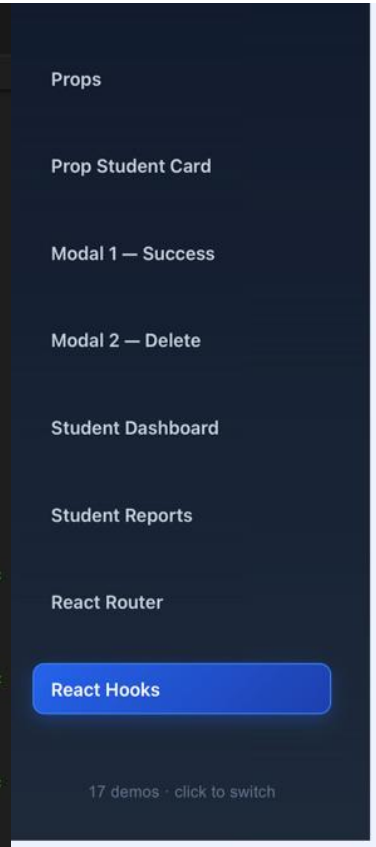
- Reusable logic.
- Cleaner components.
- Better code organization.

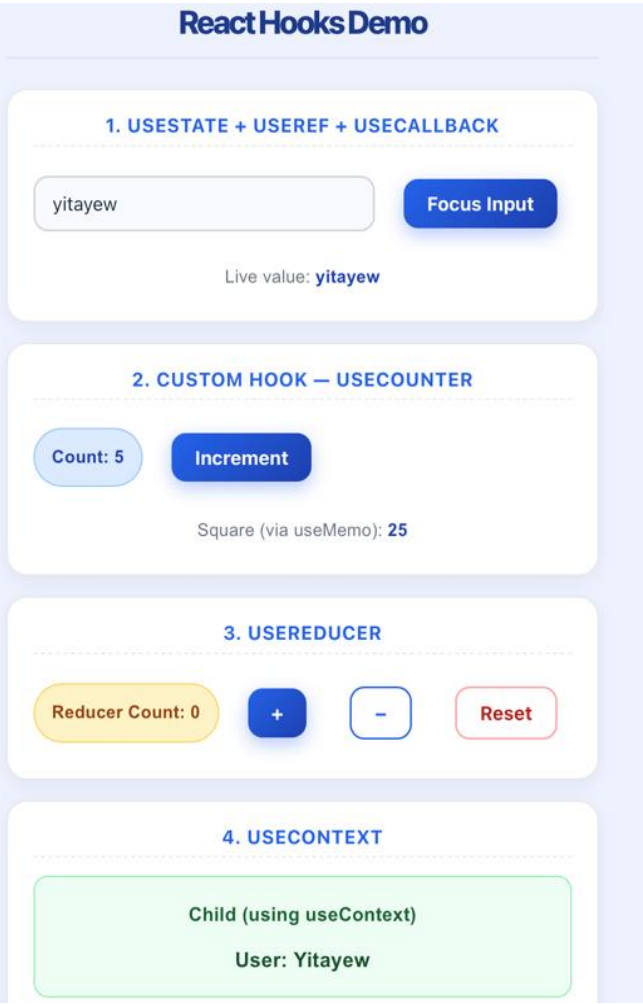
Example

App.jsx M Hook.jsx U

React_App > src > Component > Hook.jsx > ...

```
23  import React, {
24    createContext,
25    useState,
26    useEffect,
27    useContext,
28    useRef,
29    useMemo,
30    useCallback,
31    useReducer,
32  } from "react";
33
34  import "./Hook.css";
35
36  // ============================================================
37  // CONTEXT — shared value available to any descendant without
38  // prop-drilling. Here it stores the current user's name.
39  // ============================================================
40  const UserContext = createContext();
41
42  // ============================================================
43  // CUSTOM HOOK — useCounter
44  //
45  // A reusable piece of stateful logic. Its name starts with
46  // "use" (React rule for custom hooks). Any component can call
47  // it and get its own independent counter + increment function.
48  // ============================================================
```






Thank you
GRATEFUL